BY,
R.VIDISHA

LEVEL 1 :- case 1 to case 11 ⟶ scan insertion

LEVEL 2 :- case 1 to case 11 → EDT, ATPG, simulation

LABS ORDER :-

case 8, 6, 7, 9, 11, 1, 4, 3, 5, 2, 10

Simple testcases → max 100 ffs

no separate fn clk and SCLK ⇒ no OCC insertion

(i) Source the mentor graphics tool :-

source /home/tools/mentor/cshrc_mentor

↳ This command is already written in the .cshrc file. .cshrc will automatically get sourced everytime we open our terminal.

∴ MG tool will also get sourced automatically

∴ No need to type the command everyday.

(ii) Invoke the mentor graphics tool :-

To enter into the tool environment, we need to invoke the MG tool

/home/tools/mentor/Tessent/bin/tessent -shell

X

I/P files required for scan insertion:-

(i) Design i/p file :- Gate level netlist provided by the synthesis team

(ii) Library file :- contains the functionality of each and every cell which is there in our design.

Files need to be created by us:-

(i) dofile :-

write all the commands which we want to communicate to the tool.

NOTE :- We can directly communicate to the tool in the tool env itself.

But if we write the commands in a dofile, if suppose we want to make some small change, we can easily make it in the dofile and run the dofile.

(ii) logfile :- whatever we have done in the tool environment, it will be automatically stored in the logfile.

We can write a logfile while invoking the tool:

/home/tools/mentor/Tessent/bin/tessent -shell -log <logfile name>

- invoking the tool
- -log : switch

-replace : switch

if suppose we have already created a log, by giving this switch, we are asking the tool to replace the old content with new content

---

Reports and O/Ps of scan insertion :-

REPORTS
- scan cell report
- scan chain report

OUTPUTS
- scan inserted netlist
- atpg dofile
- atpg testproc
- scandef

SCAN INSERTION FLOW :-

SETUP :-

(i) providing the i/p files and lib file

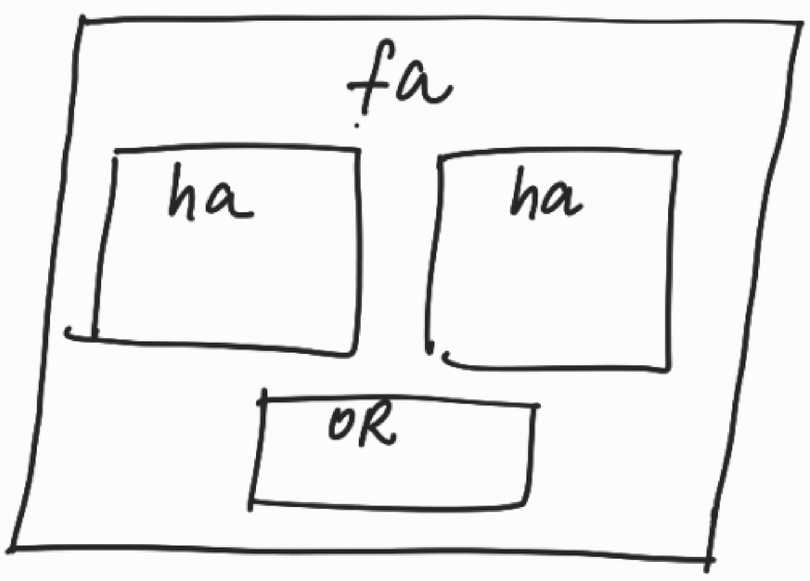
(ii) Elaboration

↳ check whether for all modules module defn is available or not.

↳ check whether there is any problem in the module instantiation

---

NOTE :-



Suppose, if ha's module defn is missing

↳ missing module defn.

```
module ha (a, b, s, ca);
    ...
    ...
endmodule
```

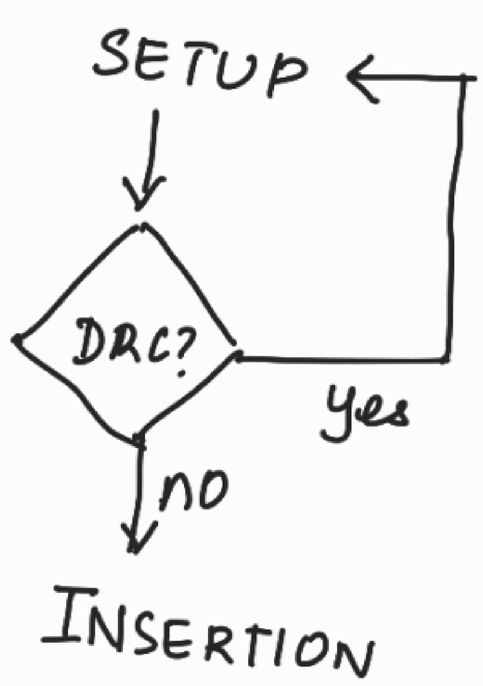
```
module fa (a, b, cin, s, cout);
    ...
    ha a1 (.a(a), .b(b), .s(n1));
```

↳ wrong module instantion

(iii) provide the declaration and constraints.

ANALYSIS : check the DRCs (Design Rule checks)

a set of rules which every flop in the design has to pass inorder to be scannable.



INSERTION : (i) identify the scan equivalent

(ii) replace all the normal ffs with scan ffs & do the scan chain stitching.

(iii) write out the reports & O/Ps.

CASE 8 :-

S1 VIOLATION :-

- When clock, set/reset declaration is wrong/missing
- Whenever the clock or set/reset of a flop is controlled by another flop or any combo logic's o/p.
- When set/reset of a flop is tied to a fixed value

S2 VIOLATION :-

- When clock of a flop is tied to a fixed value

D5 VIOLATION :- (NON SCAN MEMORY ELEMENTS)

- If a flop has S1 or S2 violation
- If a flop's scan equivalent is not available in the lib file
- If it is latch

C7 violation :- state the flop doesn't have the capability of capture

- If clock of a flop is tied to a constant value or if it is not declared.

NOTE :-

30 ffs → F1 to F15 ⇒ A CLOCK

→ F16 to F30 ⇒ B CLOCK

S1 : 30

D5 : 30

C7 : 30

→ S1-1 ⇒ A CLOCK

S1-10 ⇒ A CLOCK

S1-12 ⇒ A CLOCK

⋮ SETUP

DECLARED A CLOCK

⋮ ANALYSIS

S1 : 15

D5 : 15

C7 : 15

→ S1-1 ⇒ BCLOCK

S1-15 ⇒ BCLOCK

⋮ SETUP

DECLARE B CLOCK

⋮ ANALYSIS

NO DRC.